Original lecture content:
LEXICAL ANALYSIS
TOPIC 2

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
REMEMBER?
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Compiler pipeline: Source language -> Scanner (lexical analysis) -> tokens -> Parser (syntax analysis) -> syntax tree -> Semantic Analysis / IC generator -> intermediate language -> Code Optimizer -> Code Generator -> target language. Symbol Table has cross-stage connections to scanner, parser, semantic analysis, optimizer and code generator.

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
ROLE OF LEXICAL ANALYZER
To read the input
characters of the
produce as output a
group them into
sequence of tokens
lexemes
for each lexeme in
the source program.
source program
▪
It has to interact with Symbol Table often.
▪
When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter
that lexeme
into symbol table.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Input source characters -> group into lexemes -> produce a sequence of tokens. When an identifier lexeme is recognized, the lexical analyzer interacts with the symbol table.

Original lecture content:
INTERACTIONS BETWEEN LEXICAL ANALYZER AND PARSER.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Source program -> Lexical Analyzer --token--> Parser -> semantic analysis. Parser --getNextToken--> Lexical Analyzer. Both Lexical Analyzer and Parser interact with Symbol Table.

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
▪Simpler Design – This separation leads to simpler design of parser as unnecessary tokens (comments & whitespaces) are eliminated by the scanner already.
▪Improved Portability – The compiler's portability is enhanced as specialized symbols and characters (machine and language specific) are isolated during this phase.
▪Increased Efficiency – A separate lexical analyzer
LEXICAL
ANALYSIS
VERSUS PARSING
allows us to apply specialized techniques that serve only the lexical task, not the job of parsing. The specialized buffering techniques for reading input characters can speed up the compiler’s efficiency.

Original lecture content:
LEXICAL ANALYZER PROCESS
A. Scanning
• the simple processes
• do not require tokenization of the input
• e.g. deletion of comments and compaction of consecutive whitespace characters into one.
B Lexical analysis
• the more complex portion
• produces tokens from the output of the scanner.

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
TERMS IN LEXICAL ANALYSIS
Token:
A pair consisting of token name and an optional attribute value.
Remember the form <token-name, attribute-value>
Pattern:
An informal description of a form that lexemes of a token may take. For e.g. if token is a keyword , then pattern is just the sequence of characters that form the keyword.
Lexeme:
It is an actual stream of characters (in the source program) that matches the pattern and is identified by the lexical analyzer as an instance of that token.

Original lecture content:
Consider the following C statement:
printf(“Total= %d\n”, score);
LEXICAL
ANALYSIS
Lexeme Token
i- printf keyword
ii- score id
iii- “Total=%d\n”
literal

Original lecture content:
EXAMPLES OF TOKENS
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Table columns: TOKEN | INFORMAL DESCRIPTION | SAMPLE LEXEMES. Rows: if | characters i,f | if; else | characters e,l,s,e | else; comparison | < or > or <= or >= or == or != | <=, !=; id | letter followed by letters and digits | pi, score, D2; number | numeric constant | 3.14159, 0, 6.02e23; literal | characters surrounded by double quotes | "core dumped".

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
THE TOKEN NAMES AND ASSOCIATED ATTRIBUTE VALUES
Consider a Fortran statement:
The token names and associated attribute
values are the following:
E = M * C * * 2
<id, pointer to symbol table entry for E>
<assign-op>
<id, pointer …….. For M>
<mult-op>
▪
This lexical analyzer is performing Identifier Pattern Matching.
▪
As E=M*C**2 is input to the scanner, it starts analyzing the
stream in a sequence.
▪
Upon encountering ‘E’, the scanner recognizes and classifies it
as ‘Identifier’.
<id, pointer…… for C>
<exp-op>
<number, integer value 2>

Original lecture content:
TOKEN AND LEXEMES
Example:
Lexemse (Characters) Roles (Token-Names)
If Keyword
if(b==0)a=b
(
Parentheses
b Identifier
== Boolean Operator
0 Number
)
Parentheses
a Identifier
= Assignment Operator
b Identifier

Original lecture content:
TOKEN AND LEXEMES
Example:
Characters Roles
Const Keyword
pi Identifier
= Assignment Operator
3.1416 Number
const pi=3.1416;
Tokens formed:
<Keyword>
<id, pointer to entry for pi>
<assign-op>
<number, floating point 3.1416>

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
▪Most of the times, the lexical analyzer is not able to detect source-code errors.
▪It usually needs aid of other components to perform error-detection.
▪For example: if the source-code (in C language)
HANDLING
LEXICAL ERRORS
encounters ‘fi’, the lexical analyzer will not detect it as an error and can’t tell whether it is a misspelled keyword or an undeclared literal.
▪Since fi is a valid lexeme for the token id, the lexical analyzer must return the token id to the parser and let some other phases (probably the parser in this case) handle an error due to the transposition of letters.

Original lecture content:
HANDLING LEXICAL ERRORS
What Errors A Lexical Analyzer Can Recognize?
When none of the patterns for tokens matches a prefix of remaining input.
HANDLING
LEXICAL ERRORS
Panic Mode of Error Recovery
- The errors due to which lexical analyzer is unable to proceed with compilation, it resorts to panic mode of error recovery.
- Delete the successive characters from the remaining input until analyzer can find a well-formed token.

Original lecture content:
Possible Error Recovery Actions:
▪Deleting extraneous characters
▪Replacing or transposing two adjacent characters
▪Replacing an incorrect character by a correct one
▪Inserting missing characters
HANDLING
LEXICAL ERRORS
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
▪Lexical Analyzer often needs to look one or more characters beyond the next lexeme before we can be sure we have the right lexeme.
▪Since the large source programs consist of a large
INPUT
BUFFERING
number of characters and a huge time is required to process those characters, therefore to reduce this overhead, specialized buffering techniques have been developed.

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
SINGLE VS TWO-BUFFER
Single Buffering
Double Buffering/ Two- Buffer
▪The compiler loads a substantial segment of
▪The lexical analyzer processes characters
the source file into a buffer.
from one buffer
▪The lexical analyzer processes the buffer
▪The second buffer is simultaneously loaded
character by character to detect tokens.
with the next set of characters from the
source file.
▪Once the buffer is depleted, it is refilled with
the next set of characters, and the cycle
continues.
▪This concurrent processing and reading
ensure a steady stream of characters and
minimizes wait times.
▪This approach can be inefficient as it
requires processing to halt each time the
buffer is replenished.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Input buffering comparison: single buffer versus two-buffer arrangement. Buffer use is related to repeated character reads during lexical scanning.

Original lecture content:
INPUT BUFFERING
Two-Buffer Scheme is discussed here in which the two buffers are alternately reloaded.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Two-buffer scheme: input is divided into two alternating buffer halves. The forward pointer can cross a half-boundary and trigger a refill.

Original lecture content:
▪Size of each buffer is usually N, where N is the
size of a disk block.
▪Using one system read command, we can read N
INPUT
BUFFERING
characters into the buffer, rather than using one
system call per character.
▪Eof marks the end of source file and is different
from any possible character of the source
program.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Buffer-pair relationship: two adjacent input buffer halves are used alternately for scanning.

Original lecture content:
INPUT BUFFERING
▪Two pointers of the input are maintained:
▪lexemeBegin – marks the beginning of the current lexeme.
▪forward – scans ahead until a pattern match is found.
▪Once the next lexeme is determined, the forward is set to the character at its right end. Then, after the lexeme is recorded as an attribute value of a token returned to the parser, lexemeBegin is set to the character immediately after the lexeme just found.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Two pointers: lexemeBegin marks the start of the current lexeme; forward scans ahead to locate its end.

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
SENTINELS
▪Each time we move forward, we must check that we have not moved off one of the buffers. Because if we do, we must reload the other buffer.
▪Thus for each character read, we make two tests: one for the end of buffer and one to determine what character is read.
▪The buffer end test can be combined with the test for the current character if we extend each buffer to hold a sentinel character at the end.
▪Sentinel is the special character that cannot be a part of the source program and the natural choice is eof.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Sentinel marks the end of a buffer half; when encountered, check whether to refill or terminate.

Original lecture content:
SENTINELS
▪Note that eof retains its use as a marker for the end of entire input.
▪Any eof that appears other than at the end of buffer means that the input is at an end.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
EOF sentinel can represent an end-of-buffer boundary or true end-of-input depending on buffer position and refill result.

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
SPECIFICATION OF TOKENS
▪We need a formal way to specify or express tokens
▪Regular Expressions (Context – Free Grammar –CFG) present an important notation style for expressing the lexeme patterns or tokens.
▪Although, regular expressions cannot express all possible patterns, but they are very effective in specifying those types of patterns that we actually need for tokens.

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
1. Alphabet
It represents any finite set of symbols.
They may be letters, digits, punctuation.
e.g. {0,1} is a binary alphabet. (The alphabet consisting of
binary digits only.)
STRINGS AND
LANGUAGES
ASCII is a wonderful example of an alphabet that has many
symbols and is used in a number of software systems.
Uni code is one more example that includes 100,000
characters from alphabets around the world and represent
an important example of an alphabet.

Original lecture content:
2. String
A string over an alphabet is a finite sequence of
symbols drawn from that alphabet.
STRINGS AND
Length of a string ‘s’ (denoted by |s|) is basically the
number of occurrences of symbols in it.
LANGUAGES
E.g. the string ‘kite’ has a length of 4.
The empty string denoted by ‘ε’ has Zero length.

Original lecture content:
STRINGS AND
LANGUAGES
The commonly used
string related terms
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
STRINGS AND
3. Language
LANGUAGES
A countable set of strings over some fixed alphabet.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
SUMMARY
- An alphabet ∑ is a set of characters
Example: ∑ = {a, b}
- A string over ∑ is a finite sequence of elements from ∑
Example: aabba
- A language is a set of strings
Example: L = {a, b, abab, babbba}
- Regular languages: a subset of all languages that can be defined by regular expressions

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
OPERATIONS ON LANGUAGES
The most fundamental operations on languages are:
▪Union
▪Concatenation
▪Closure

Original lecture content:
OPERATIONS ON LANGUAGES
If x and y are two strings then, concatenation is denoted by xy (the string formed y appending y to x).
For example, if x=dog and y=house then xy=doghouse.
The empty string is identity under concatenation i.e. for any string s, □s=s □ =s

Original lecture content:
OPERATIONS ON LANGUAGES
Let L be the set of letters {A; B; …; Z; a; b; …; z}
and let D be the set of digits {0; 1; …9}.
a)
L ∪ D
b)
LD
c) □4
d) □*
e) □□∪□
f)
□+

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
▪A regular expression is a sequence of
characters that defines a search pattern,
typically used for pattern matching with strings.
▪Commonly used in lexical analysis to define the
REGULAR
EXPRESSIONS
patterns for tokens in a language.
▪Example:
Let ∑ = { Letter, Digit }
Letter = { A, B, C, ----, Z }
Digit = { 1, 2, 3, 4, ----, 9 }
Then the regular expression will be:
letter (letter | digit)*
(RE)

Original lecture content:
RECONSTRUCTION
Regular expressions are built recursively from smaller expressions.
Each regular expression r denotes a language L(r), which is also definned recursively from the languages denoted by r's subexpressions.
BASIS:
There are two rules that form the basis:

- □ is a regular expression, and L(□) is {□}, that is, the language whose sole member is the empty string.
- If a is a symbol in ∑, then a is a regular expression, and L(a) = {a}, that is, the language with one string, of length one, with a in its one position.

Original lecture content:
REOPERATOR
(r | s) denotes L(r) ∪L(s)
r or s (alternation)
(rs) denotes L(r)L(s)
r followed by s (concatenation)
(r*) denotes (L(r))*
Zero or more occurrences of r (kleen closure)
(□+) denotes (L(r))+
One or more occurrences of r (Positive Closure)
(r)
Used for grouping

Original lecture content:
PRECEDENCE FOR RE OPERATORS
RE Operators
Analogous Arithmetic Operator
Precedence
x|y
x + y
lowest
xy
x * y
middle
x* , x+
x ^ y
highest
Spot the difference
Letter letter | digit *
vs
Letter (letter|digit) *

Original lecture content:
EXAMPLES OF REGULAR EXPRESSIONS
Let ∑ = {a, b})

- a|b : Denotes the language {a, b}.
- (a|b)(a |b) : Denotes {aa, ab, ba, bb}, the language of all strings of length 2 over the alphabet ∑. Equivalent to: aa|ab|ba|bb.
- a*: Denotes the language consisting of all strings of zero or more a's, {□, a, aa, aaa, ...}
- (a|b)* : Denotes all strings consisting of zero or more a’s or b’s.
- Example strings: {□, a, b, aa, ab, ba, bb, aaa, ...}.Equivalent to: (a*b*)*.
- a|a*b : Denotes the language {a, ab, aab, aaab, ...}, i.e., the string a followed by zero or more a’s and ending with b

Original lecture content:
ANOTHER EXAMPLE
Consider the alphabet ∑ = { a, b, c } and the language is :
L = { a, c, ab, cb, abb, cbb, abbb, cbbb, abbbb, cbbbb }
L = { All words of a’s, b’s and c’s that either starts with a or c followed by any number of b’s}
RE will be:
(a | c) b* or (a + c) b*

Original lecture content:
TOPICS
Role of Lexical Analysis
• Lexical Analysis Versus Parsing
• Tokens, Patterns, and Lexemes
• Attributes for Tokens
• Lexical Errors
Input Buffering
• Buffer Pairs
• Sentinels
Specification of Tokens
• Strings and Languages
• Operations on Languages
• Regular Expressions
• Regular Definitions
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
REGULAR DEFINITIONS
Regular definitions allow names to be given
to certain regular expressions. These names
can then be used in subsequent expressions
as if they were symbols.
If ∑ is an alphabet of basic symbols, then a
regular definition is a sequence of definitions
of the form:
d1→r1
d2 → r2
…
dn → rn
where:
1. Each □□ is a new symbol, not in ∑ and
not the same as any other of the d’s
2. Each □□ is a regular expression over the
alphabet ∑υ{□1, □2, …, □□−1},

Original lecture content:
REGULAR DEFINITIONS EXAMPLE
C identifiers are strings of letters, digits, and underscores
Simplified version:
Regular definitions:

Original lecture content:
REGULAR DEFINITIONS EXAMPLE
Unsigned numbers (integer or floating point) are strings such as 5280, 0.01234, 6.336E4, or 1.89E-4.
Simplified version:
Regular definition:

Original lecture content:
LEXICAL ANALYSIS – PART 2
TOPIC 2
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
TOPIC
Recognition of tokens
Transition Diagrams
Recognition of reserved words and identifiers
Architecture of a Transition-diagram- Based Lexical Analyzer
Finite Automata
• Key components
• NFA VS DFA
• Minimization of DFA
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
LET’S RECALL
Patterns for tokens
Tokens, their patterns and attribute values
Simple form of branching statements and
conditional expressions in Pascal form
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Original lecture content:
Classify program substrings according
to role
WHAT ARE
TOKENS
Output of lexical analysis is a stream of
tokens . . .which is input to the parser
FOR?
Parser relies on token distinctions
An identifier is treated differently than a
keyword

Original lecture content:
Symbols: +, -, *, /, =, <, >, ->,
…
Keywords:
if, while, struct,
float, int, …
TYPICAL
TOKENS IN
Integer and Real (floating point)
literals
123, 123.45
A PL
Char (string) literals
Identifiers
Comments
White space

Original lecture content:
TRANSITION
DIAGRAMS
Consists of:
1. A collection of nodes or circles, called states.
Each state represents a condition that could occur during the process of scanning the input looking for a lexeme that matches one of several patterns
2. Edges are directed from one state of the transition diagram to another.
Each edge is labeled by a symbol or set of symbols.
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
Transition diagrams represent lexical recognition as directed states and input-labeled transitions. Acceptance is indicated by designated final states.

Original lecture content:
TRANSITION DIAGRAM FOR RELOP
relop = < | <= | = | <> | > | >=
=
return(relop,LE)
return(relop,NE)
>
<
start
other
*
=
return(relop,LT)
return(relop,EQ)
>
=
return(relop,GE)
other
*
return(relop,GT)
* indicates retract the input one position
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Exact state IDs, transition labels and accepting states in the relational-operator transition diagram require manual verification against the slide.]

Original lecture content:
RECOGNITION OF IDENTIERS
ID = letter(letter | digit) *
letter or digit
other
*
start
letter
return(id)
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Exact identifier-recognition transition graph and retract conditions require manual verification.]

Original lecture content:
MAPPING TRANSITION DIAGRAMS INTO C CODE

```
switch (state) {
case 9:
if (isletter( c) )
state = 10;
else
state = failure();
break;
case 10:
c = nextchar();
if (isletter( c) || isdigit( c) )
state = 10;
else
state 11;
break;
case 11:
letter or digit
start
letter
return(id)
other
retract(1);
insert(id);
return;
}
```

Original lecture content:
LEXICAL ANALYZER LOOP

```
Token nexttoken() {
while (1) {
switch (state) {
case 0:
c = nextchar();
if (isspace(c))
state = 0;
else if (c == '<')
state = 1;
else if (c == '=')
state = 5;
break;
case 9:
c = nextchar();
if (isletter(c))
state = 10;
else
state = fail
break;
case 10:
// Logic for state 10 goes here
break;
case 11:
retract(1);
insert(id);
return;
}
}
}
```

Original lecture content:
RECOGNITION OF RESERVED WORDS
▪Install the reserved words in the symbol table initially.
▪A field of the symbol-table entry indicates that these strings are never ordinary identifiers, and tells which token they represent.

Original lecture content:
THE TRANSITION DIAGRAM FOR TOKEN NUMBER
Accepting integer
Accepting float
e.g. 12.31
Accepting float
e.g. 12.31E4
e.g. 12
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Numeric-token transition graph contains state transitions whose exact labels require manual verification.]

Original lecture content:
END OF TOPIC 2
CPT316 PROGRAMMING LANGUAGE IMPLEMENTATION AND PARADIGMS